* Assignment - 3 *

Q.1 Explain Mutex & semaphore concepts in detail.

→ Mutex (Mutual Exclusion)

A Mutex is like a lock:

- Only one thread can own (lock) the mutex at a time.

- Other threads that want to access the same resource must wait until the mutex is released.

→ Used when only one thread at a time should access the resource.

Key points.

- Binary object: It has two states - locked (1) or unlocked (0).

- A thread acquires (locks) the mutex before accessing the shared resource.

- once done, the thread releases (unlocks) the mutex.

* Example Analogy

- Imagine a washroom with a single key (mutex)
- If one person has the key, others must wait until it is returned.

Date
Page

2) Semaphore

A semaphore is a more general synchronization tool, often described as a counter that manages acess to resources.

→ used when there are multiple instances of a resource available.

Type of semaphores.

1) Binary Semaphore.
- works like a mutex (value ~~or~~ 0 or 1).
- But difference: mutex has ownership, while a semaphore doesn't enforce ownership.

2) Counting semaphore.
- Value can be greater than 1.
- The value represent how many resources are available.
- when a thread acquires a resource, the semaphore is decremented.

Ex :-
- imagine a parking lot with 5 spaces.
- each car arriving dec. the count by 1.

Q.2 what is the Dining philosophers problem?

→ The Dining philosophers problem is a classic synchronization & concurrency problem in computer science, introduced by Edsger Dijkstra in 1965. It illustrates issue related to resource allocation, Deadlock, & starvation in a multi-process system.

* problem Setup.

- Imagine five philosophers sitting around a circular dining table.

- Each philosopher has one plate of food in front of them.

- B/w each pair of philosophers, there is one fork (so 5 forks in total).

- philosophers follow a cycle:
  a. think (not hungry, just reasoning).
  b. Get hungry (decide to eat)
  c. Try to pick up two forks (one at a time).
  d. Eat.
  e. put down both forks.
  f. Go back to thinking.

* The problem
The challenge arises when philosophers attempt to pick up forks :

1) Deadlock :- If every philosopher picks up their left fork first & then waits for the right fork, no one will ever get both forks.

* Key issue modeled.
- Deadlock (circular waiting for resources)
- Starvation (a process never gets a resource)
- Resource contention (multiple process competing for limited resources).

* Solutions.
Several strategies exist to solve or mitigate the problem :
1) Asymmetric resource picking :
- odd-numbered philosophers pick up left fork first, then right.
- Even-numbered philosophers pick up right fork first, then left.

2) Limit concurrency :
- Allow only four philosophers to sit at the table at once, ensures at least one can eat.

Q.3 what are the necessary cond^n for a deadlock in the system?

→ * Necessary cond^n for Deadlock.

1) Mutual Exclusion

- At least one resource must be held in a non-sharable mode (only one process can it use it at a time).
- example :- A printer can't be shared by two processes at the same time.

2) Hold & wait.

- A process must be holding at least one resource & simultaneously waiting to acquire additional resources that are currently being held by other process.

3) No preemption

- Resource cannot be forcibly taken away from a process; they must be released voluntarily by the process after completion.

ex) you can't just take the printer from P1 unless it releases it.

4) Circular wait.

- A circular chain of processes exists, where each process is waiting for a resource held

by the next process in the chain.

- Example : P1 → waiting for P2's resource, P2 → waiting for P3's resource, - - - - Pn → waiting for P1's resource

Q.4 Explain Banker's Algorithm with an example

→ Banker's Algorithm is a deadlock avoidance algorithm used in operating systems. It checks whether allocating requested resources to a process will keep the system in a safe state or lead to deadlock.

it's Called "Banker's Algorithm" because it works like a banker who gives out loans but ensures he always has enough money so everyone can eventually be repaid.

- Key Terms.
  - Available → the number of instance of each resource currently free.
  - max → maximum demand of each process for resources.
  - Allocation → Resource currently allocated to each process.
  - Need = max − Allocation → Remaining resources needed by each process.

- Banker's Algorithm Steps.

1) Check request validity :-
   - If request ≤ need & request ≤ available, proceed, otherwise, reject.

2) Pretend allocation : Temporarily allocate requested resources & update available, Allocation, & Need.

3) Safety check : Use safety Algorithm to see if the system is still in a safe state.
   - If yes → Grant request.
   - If no → Rollback (don't grant request)

*

Q.5 Example :- Considering a system with five processes P_0 through P_4 & three resources of type A, B, C. Resource type A has 10 instance, B has 5 instances & type C has 7 ist instances. Suppose at time t_0 following snapshot of the system has been taken:

Process	Allocation A	Allocation B	Allocation C	Max A	Max B	Max C	Available A	Available B	Available C
P_0	0	1	0	7	5	3	3	3	2
P_1	2	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	2	1	1	2	2	2			
P_4	0	0	2	4	3	3			